

ICS 课程实验报告 (Lab 0)

—— Git 版本控制与 GitHub 协作实践

课程名称：ICS (Introduction to Computer Systems) 实验题目：Lab 0 Git 与 GitHub 基础操作

姓名：苏战彪 学号：25301030062 实验环境：WSL Ubuntu + GitHub

远程仓库：github.com/A-fool-sleeping/huziTestLab

一、实验目的

通过本次实验，熟悉分布式版本控制系统 Git 的基本工作流程，掌握仓库克隆、文件修改、暂存 (add)、提交 (commit) 与推送 (push) 等核心命令；理解分支 (branch)、合并 (merge) 与冲突解决的概念；了解业界主流的 Commit Message 规范与分支管理工作流 (Git Flow)，为后续课程实验与团队协作开发打下基础。

二、实验环境

操作系统：Windows + WSL (Ubuntu 子系统)；版本控制：Git；远程托管：GitHub；代码编辑：nano 命令行编辑器；远程仓库基于课程 TestLab 模板通过 Use this template 生成，名为 huziTestLab。

三、任务一：思考题

1. 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

此前在课程小组作业与项目实践中，曾有过简单的多人协作经历。当时主要通过网盘、即时通讯工具直接互传文件，每人负责不同模块，最后由一人手动合并代码。这种方式没有版本记录，文件命名混乱（如 final、final2、最终版真的最终版），一旦他人改动了同一处代码，很难追溯是谁、在什么时间、改了什么，也难以回退。本次实验使用 Git 之后，才体会到版本控制工具对多人协作的真正价值。

2. Git 为什么要设计“暂存 (add) —提交 (commit)”两个步骤？

Git 在工作区与版本库之间引入了“暂存区 (staging area / index)”作为中间缓冲区。工作区是正在编辑的文件，commit 只会把暂存区中的内容打包成一次提交快照。这样设计的好处是：开发者可以在一次开发中同时修改多个文件，再通过 git add 有选择地把相关改动挑进暂存区，把“完成同一件事”的改动组织成一次语义清晰的提交，而不必把所有杂糅在一起的修改一股脑提交；也方便在提交前用 git status 与 git diff --cached 复查将要提交的内容，使提交历史更干净、可追溯。

3. git branch 和 git branch -a 的区别是什么？

git branch 仅列出本地仓库中存在的分支；加上 -a (即 --all) 参数后，会同时列出本地分支与远程仓库上所有被本地追踪到的远程分支（以 remotes/<■■■■>/<■■■■> 形式显示）。因此 git branch -a 能更完整地反映本地与远程的分支全貌。

四、任务二：修改 main.c 并提交推送

在本地通过 SSH 方式克隆个人仓库 huziTestLab，进入目录后使用 nano main.c 编辑文件，将原 printf("Hello, world!\n") 修改为 printf("I have done!\n")，完成 TODO。随后依次执行：

```
git status (查看工作区改动)
git add main.c (将修改加入暂存区)
git commit -m "Lab0 task2: complete TODO in main.c" (本地提交)
git push origin main (推送到远程 main 分支)
```

执行结果如下图所示：终端显示 main.c 被修改 (modified)，提交记录显示 1 file changed, 1 insertion(+), 1 deletion(-)，推送成功后输出 main -> main。

```
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ nano main.c
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ git add main.c
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ nano main.c
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   main.c

no changes added to commit (use "git add" and/or "git commit -a")
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ git add main.c
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ git commit -m "Lab0 task2: complete TODO in main.c"
[main 2137f10] Lab0 task2: complete TODO in main.c
 1 file changed, 1 insertion(+), 1 deletion(-)
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 300 bytes | 300.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:A-fool-sleeping/huziTestLab.git
 ccabae6..2137f10  main -> main
```

图 1 任务二：编辑 main.c、add / commit / push 全过程

五、任务三：资料阅读与思考

1. 《Commit message 和 Change log 编写指南》（阮一峰）内容概括

该文介绍了业界广泛采用的 Angular 风格 Commit Message 规范。一次完整的提交信息分为 Header（必填）、Body（可选）、Footer（可选）三部分，其中 Header 格式为 <type>(<scope>): <subject>。type 用于说明提交类别，常见取值有：feat（新功能）、fix（修复 bug）、docs（文档变更）、style（格式调整，不影响代码逻辑）、refactor（重构）、perf（性能优化）、test（测试）、chore（构建/工具变更）。subject 应使用动词原形、简短、不加句号。Body 详细描述动机与改动，Footer 用于标注不兼容变更或关闭 issue。遵循该规范后，可借助工具（如 commitizen、conventional-changelog）自动生成结构化的 Change Log。

2. 《Git Flow 分支管理工作流》内容概括

Git Flow 是一套面向版本发布的分支管理模型，为不同用途约定了固定分支：master 存放生产环境上的稳定发布版本并打 tag；develop 是日常开发的主集成分支；feature/* 从 develop 切出用于开发新功能，完成后合回 develop；release/* 用于发布前的测试与 bug 修复；hotfix/* 从 master 切出处理线上紧急问题，修复后同时合回 master 与 develop。其核心原则是“分支从哪里切出，最终合并回哪里”，从而保证稳定版本不被未完成的功能污染，适合中大型团队的并行开发与版本迭代。

3. 为什么要学习 Git

Git 是当前软件开发领域事实上的版本控制标准，学习 Git 的意义主要体现在：第一，版本可回溯：每一次 commit 都是一份项目快照，改错了可以随时回退，不再需要手动保存“最终版 v1/v2/真的最终版”这类冗余备份；第二，分支支持并行开发：可以新建分支试验新想法，而不破坏主分支的稳定代码，配合 Git Flow 等策略能让多人分工井然有序；第三，支持多人协作：开发者在本地独立工作，再通过 push / pull 合并到远程仓库，冲突时人工甄别解决，每行代码的修改者与时间都可追溯，便于代码评审；第四，过程规范化：配合规范的 commit message，项目历史清晰可读，还能自动生成更新日志，方便后期维护与问题定位；第五，行业通用技能：从课程作业、竞赛项目到未来的工程实习，代码托管、协作与开源贡献都以 Git 为基础，是计算机相关专业学生的必备基本功。

六、任务四：分支创建、合并与冲突解决

本任务练习分支的创建、切换、提交与合并。操作流程如下：

```
# 在 main 分支修改 main.c 并提交
git add main.c
```

```
git commit -m "task4: modify main.c on main branch"
# 创建并切换到 feature 分支
git checkout -b feature
# 在 feature 分支再次修改同一行 printf 并提交
git add main.c
git commit -m "task4: modify main.c on feature branch"
# 切回 main，合并 feature，触发冲突
git checkout main
git merge feature
```

由于 main 分支与 feature 分支都修改了 main.c 中 printf 所在的同一行，Git 无法自动判断保留哪一版，于是报告 CONFLICT (content): Merge conflict in main.c。此时打开 main.c，会看到 <<<<<<< HEAD、=====、>>>>>>> feature 三组冲突标记；手动编辑保留最终想要的代码行、删除全部冲突标记后，再执行 git add main.c 与 git commit -m "Task4: resolve merge conflict"，完成合并并 git push origin main 推送到远程。

```
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ nano main.c
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ git add main.c
git commit -m "task4: modify main.c on main branch"
[main aea8423] task4: modify main.c on main branch
1 file changed, 1 insertion(+), 1 deletion(-)
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ git checkout feature
Switched to branch 'feature'
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ nano main.c
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ ^C
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ git add main.c
git commit -m "task4: modify main.c on feature branch"
[feature eed96ac] task4: modify main.c on feature branch
1 file changed, 1 insertion(+), 1 deletion(-)
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ git merge feature
Already up to date.
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 3 commits.
(use "git push" to publish your local commits)
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ nano main.c
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ git add main.c
git commit -m "Task4: resolve merge conflict"
[main 2680699] Task4: resolve merge conflict
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ git push origin main
Enumerating objects: 20, done.
```

图 2 任务四：分支提交、合并冲突产生与解决过程

```
jcc@LAPTOP-Q6JEH4VM:~/huziTestLab$ git push origin main
Enumerating objects: 20, done.
Counting objects: 100% (20/20), done.
Delta compression using up to 12 threads
Compressing objects: 100% (16/16), done.
Writing objects: 100% (16/16), 1.46 KiB | 498.00 KiB/s, done.
Total 16 (delta 10), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (10/10), completed with 2 local objects.
To github.com:A-fool-sleeping/huziTestLab.git
2137f10..2680699 main -> main
```

图 3 冲突解决后推送 main 分支成功

七、实验总结

通过本次 Lab 0，完整走通了“克隆仓库 → 修改文件 → 暂存 → 提交 → 推送”的基本循环，并亲手经历了一次分支合并冲突的产生与解决过程。实践中也遇到了一些典型问题：例如把 git config --global 误拆成两条命令导致 global: command not found；未生成 SSH 密钥就查看 id_rsa.pub 导致文件不存在；在 WSL 中首次运行 code 会反复下载 VS Code Server，改用 nano 后顺利完成编辑；在 feature 分支上执行 git merge feature 出现 “Already up to date”，原因是合并的是分支自身，切回 main 后再合并才真正触发冲突。这些问题让我更深入地理解了 Git 的工作区、暂存区、版本库与分支指针之间的关系，为后续课程实验打下了良好基础。